Задача 6. Банковские счета

Источник:	основная
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	256 мегабайт

Необходимо реализовать систему банковских счетов в объектно-ориентированном стиле на языке C. В системе существуют счета двух типов: дебетовый и кредитный. Все счета имеют общий интерфейс операций, но различаются правилами снятия средств.

Реализация должна использовать инкапсуляцию, разделение интерфейса и реализации, а также полиморфизм через таблицу виртуальных функций (vtable). Прямой доступ к внутренним полям структуры счёта из пользовательского кода запрещён.

Базовый тип счёта

Все счета основаны на общей структуре bank_account, которая содержит:

- account_number — номер счёта (целое число, уникальное для каждого счёта);
- balance — текущий баланс (вещественное число);
- vtable — указатель на таблицу виртуальных функций.

Таблица виртуальных функций

Таблица bank_account_vtable_t должна содержать указатели на следующие функции:

- deposit — пополнение счёта;
- withdraw — снятие средств;
- get_balance — получение текущего баланса;
- get_type — получение строкового представления типа счёта.

Файловая структура

1. bank_account.h — публичный заголовочный файл с объявлением базовой структуры и vtable.
2. bank_account_p.h — приватный заголовочный файл для внутреннего использования (доступен только реализации).
3. bank_account.c — реализация базовых операций для всех счетов.
4. debit_account.h — публичный интерфейс дебетового счёта.
5. debit_account_p.h — приватный заголовок дебетового счёта.

6. debit_account.c — реализация дебетового счёта.
7. credit_account.h — публичный интерфейс кредитового счёта.
8. credit_account_p.h — приватный заголовок кредитового счёта.
9. credit_account.c — реализация кредитового счёта.
10. test.cpp — файл с модульными тестами на GoogleTest.
11. BUILD — файл сборки для системы Bazel **(name = "bank_account_test")**!!!!.

Функциональные требования

Общие операции для всех счетов

- void deposit(bank_account_t* acc, double amount) — пополнить счёт на указанную сумму.
- int withdraw(bank_account_t* acc, double amount) — снять указанную сумму. Возвращает 1 при успехе и 0 при неудаче.
- double get_balance(bank_account_t* acc) — вернуть текущий баланс.
- const char* get_type(bank_account_t* acc) — вернуть строку с типом счёта ("Debit" или "Credit").
- void destroy_account(bank_account_t* acc) — освободить память, занятую счётом.

Дебетовый счёт (Debit Account)

Дебетовый счёт представляет собой классический банковский счёт, где клиент расходует только собственные средства.

- **Создание:** bank_account_t* create_debit_account(int number) — создаёт новый дебетовый счёт с указанным номером. Начальный баланс равен 0.
- **Пополнение:** всегда успешно, баланс увеличивается на указанную сумму.
- **Снятие:** баланс не может становиться отрицательным. Если сумма снятия превышает текущий баланс, операция завершается неудачей (возвращает 0), баланс не изменяется.
- **Тип счёта:** возвращается строка "Debit".

Кредитный счёт (Credit Account)

Кредитный счёт позволяет клиенту уходить в минус в пределах установленного кредитного лимита.

- **Создание:** bank_account_t* create_credit_account(int number, double credit_limit) — создаёт новый кредитный счёт с указанным номером и кредитным лимитом. Начальный баланс равен 0.

- **Пополнение:** всегда успешно, баланс увеличивается на указанную сумму (при отрицательном балансе сначала погашается задолженность).
- **Снятие:** баланс не может быть меньше значения -credit_limit. Если сумма снятия приводит к балансу ниже этого порога, операция завершается неудачей.
- **Тип счёта:** возвращается строка "Credit".

Особые случаи:

- Кредитный лимит = 0: счёт ведёт себя как дебетовый (не может уходить в минус).
- Отрицательный кредитный лимит: снятие средств запрещено (любая попытка снятия должна завершаться неудачей).

Обработка некорректных входных данных

- Передача NULL в качестве указателя на счёт должна обрабатываться корректно (без падения программы).
- destroy_account(nullptr) не должна вызывать ошибок.
- deposit(nullptr, amount), withdraw(nullptr, amount) должны завершаться без изменений состояния.
- get_balance(nullptr) должен возвращать 0.0.
- get_type(nullptr) должен возвращать строку "Unknown".

Файлы *.h можно получить по ссылке. Файлы *c/*cpp, файл и файл BUILD необходимо предоставить тестирующей системе (**(name = "bank_account_test")**!!!!).